[Huyen Nguyen] C Programing Review

Session 2

Status: In progress...

Section: Linux System Programming

Topic: C Programming Language Review

Duration: 3 working days (8 hours/day)

1. Pointer Arithmetic

```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;

printf("%p\n", (void*)p);
printf("%p\n", (void*)(p + 1));  /* jumps 4 bytes */
```

`(p + 1)` doesn't add 1 to the address — it adds `1 * sizeof(*p)`. The compiler automatically multiplies by the size of the data type the pointer points to.

```
char *cp = (char*)arr;
int  *ip = arr;
double *dp;

cp + 1;   /* jumps 1 byte  (sizeof(char) = 1) */
ip + 1;   /* jumps 4 bytes (sizeof(int) = 4) */
dp + 1;   /* jumps 8 bytes (sizeof(double) = 8) */
```

→ An incorrect cast will jump the wrong number of bytes and misread the memory region.

`void*` doesn't know the size of the element it points to → the addition operation isn't well-defined per the C standard.

```
void *vp = arr;
vp + 1;   /* Compile error */
```

```
int a[5], b[5];
int *p1 = &a[0];
int *p2 = &b[0];
p2 - p1;   /* UNDEFINED BEHAVIOR */
```

Can't subtract two pointers from different arrays because `a` and `b` aren't adjacent in memory.

2. `static` and `extern`

`static` on a local variable inside a function — the variable retains its value across function calls (it isn't destroyed when the function returns).

```
void counter()
{
    /* this line only initializes once, doesn't re-run */
    static int count = 0;
    count++;
    printf("%d\n", count);
}

counter();  /* prints 1 */
counter();  /* prints 2 */
counter();  /* prints 3 */
```

This variable is stored in the Data segment, with scope limited to the function

→ it does NOT live on the Stack (since the Stack is destroyed when the function returns).

`static` on a global variable/function — restricts its scope to that single file; other files that `#include` or `extern` it cannot access it.

```
/* file1.c */
/* only file1.c can see this variable */
static int internal_counter = 0;

/* only file1.c can call this function */
static void helper()
```

```
{
    ...
}
```

→ Used to simulate file-level "private" scope — avoiding name conflicts when linking multiple `.o` files together.

`extern` — declares a variable/function that is defined in ANOTHER FILE.

```
/* file1.c */
int shared_value = 100;   /* definition */

/* file2.c */
/* declaration only, doesn't request memory allocation - tells the
   compiler this variable exists somewhere and will be linked later */
extern int shared_value;

void show()
{
    /* usable, even though it's not defined in file2.c */
    printf("%d\n", shared_value);
}
```

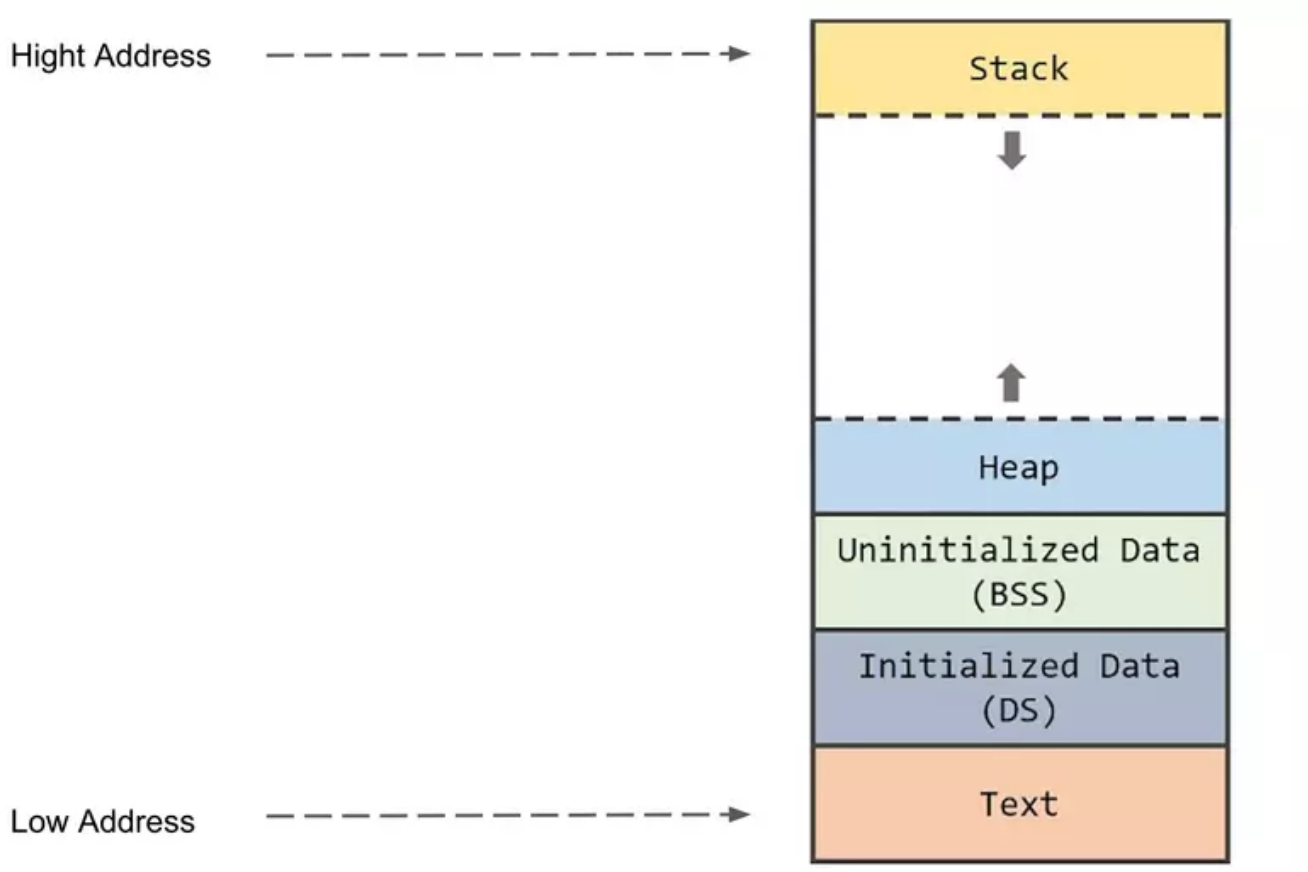
If don't use `extern` and instead re-declare `int shared_value` in file2.c

→ multiple definition, because the linker sees two memory regions allocated for the same variable name when combining the files.

-> It can cause a linker error

3. Memory Layout

When a C program runs, it's divided into memory regions, in order from low → high address:



Memory Layout Diagram

- Code/Text: contains machine code, read only
- Data: initialized global variables
- BSS: uninitialized global variables
- Heap: dynamic allocation
- Stack: local variables, return addresses

```
int global_uninit;              /* BSS (not assigned a value) */
int global_init = 5;            /* Data (assigned a value) */
static int static_uninit;       /* BSS */
static int static_init = 10;    /* Data */

int main()
{
    int local_var = 1;          /* Stack */

    /* pointer p lives on the Stack, the memory it points to lives on
       the Heap */
    int *p = malloc(sizeof(int));

    /* the string "hello" lives in the read-only region of Data, the
```

```
        pointer str lives on the Stack */
    char *str = "hello";
}
```

4. Struct Padding & Alignment

The CPU reads memory most efficiently when data is aligned to a multiple of its type's size.

The compiler automatically inserts padding (filler bytes, garbage values) to ensure this.

```
struct A
{
    char c;    /* 1 byte */
    int i;     /* 4 bytes */
};
printf("%zu\n", sizeof(struct A));   /* 8 */
```

`struct A` in memory:

```
┌───┬───┬───┬───┬───┬───┬───┬───┐
| c |pad|pad|pad|       i       |
└───┴───┴───┴───┴───┴───┴───┴───┘
  0   1   2   3   4   5   6   7
```

`c` occupies byte 0, but `i` needs to start at an address divisible by 4 → bytes 1, 2, 3 are "wasted" as padding, and `i` starts at byte 4.

Field declaration order affects the struct's size:

```
struct B
{
    char c1;   /* 1 byte */
    int i;     /* 4 bytes */
    char c2;   /* 1 byte */
};
printf("%zu\n", sizeof(struct B));   /* 12 bytes */

struct C
{
    int i;     /* 4 bytes */
```

```
    char c1;   /* 1 byte */
    char c2;   /* 1 byte */
};
printf("%zu\n", sizeof(struct C));   /* 8 bytes */
```

→ When designing a struct, fields should be ordered from largest to smallest size to reduce padding.

Forcing no padding — `packed` (GCC extension)

```
struct __attribute__((packed)) D
{
    char c;
    int i;
};
printf("%zu\n", sizeof(struct D));   /* 5 - no more padding */
```

5. Macro (`#define`)

5.1. Pitfall 1

Macros are pure text substitution (not real functions) — the root cause of every bug

```
#define SQUARE(x) x * x
int result = SQUARE(3 + 2);   /* Expected: 25, ACTUAL: 3+2*3+2 = 11 */
```

Reason: the preprocessor replaces `SQUARE(3 + 2)` with the text `3 + 2 * 3 + 2`.

The correct fix — ALWAYS wrap both the parameter and the whole macro in parentheses:

```
#define SQUARE(x) ((x) * (x))
SQUARE(3 + 2);   /* ((3+2) * (3+2)) = 25 - correct */
```

5.2. Pitfall 2 - Multi-line macros need `do { } while(0)`

```
#define SWAP(a, b) { int t = a; a = b; b = t; }

if (cond)
    SWAP(x, y);
```

```
else
    do_something();
```

After the preprocessor substitutes it, the actual code becomes:

```
if (cond)
    { int t = x; x = y; y = t; };
    /* the trailing ; after } creates an empty statement */
else
    /* "else" is now NOT attached to "if" anymore -> COMPILE ERROR */
    do_something();
```

Standard fix — wrap it in `do { ... } while(0)`:

```
#define SWAP(a, b) \
    do { int t = a; a = b; b = t; } while (0)
```

→ This structure requires a trailing `;` and works correctly in every `if/else` context.

5. Dynamic Allocation

`malloc`/`calloc` vs. using a local variable

Example: using a local variable

```
int* create_array_wrong(int size)
{
    int arr[size];       /* local variable - lives on the STACK */
    return arr;           /* returns pointer to destroyed memory */
}
```

When the function returns, its stack frame is freed → the returned pointer is a dangling pointer. Using it at the call site is UNDEFINED BEHAVIOR (it might accidentally "seem to work," but it is WRONG).

→ Data on the Heap exists independently of the function's scope, and is only lost when `free()` is called.

Example: using `malloc`

```
int* create_array_correct(int size)
{
```

```
    /* Heap - lives beyond the function's scope */
    int *arr = malloc(size * sizeof(int));
    return arr;    /* SAFE - memory still exists after return */
}
```

Question: What's the real difference between `malloc` and `calloc`?

```
int *a = malloc(5 * sizeof(int));    /* memory contains garbage */
int *b = calloc(5, sizeof(int));     /* memory = 0 */
printf("%d\n", a[0]);   /* undefined, could be a garbage number */
printf("%d\n", b[0]);   /* 0 */
```

→ `calloc` takes 2 parameters (number of elements, size of each element) and can check for integer overflow itself — safer than `malloc(n * size)` (if `n * size` overflows as an integer, `malloc` might allocate less than expected, leading to a buffer overflow).

Question 3: What happens if don't call `free()` after using the memory?

```
void leak_function()
{
    int *p = malloc(1000 * sizeof(int));
    /* ... use p ... */
    /* does NOT call free(p) before the function returns */
}
/* the pointer p on the Stack is destroyed, but the 1000-int region
   on the HEAP is NOT freed */
```

→ The program uses more RAM over time until it runs out of memory. For long-running programs (network daemons, 24/7 services) this is a serious bug — it accumulates gradually instead of crashing right away.

Detection tool: `valgrind --leak-check=full ./program`

Question 4: Use-After-Free / Dangling Pointer

```
int *p = malloc(sizeof(int));
*p = 42;
free(p);
printf("%d\n", *p);   /* UNDEFINED BEHAVIOR - may still print 42 */
*p = 100;              /* overwrite */
```

`free(p)` only tells the memory allocator that this region is no longer in use — it does NOT erase the data → `p` still holds the old address (dangling pointer).

→ If the program later calls `malloc()` again and the system reuses that memory region for a different variable, it will get overwritten.

→ Always set the pointer to NULL after freeing it.

Double Free

```
int *p = malloc(sizeof(int));
free(p);
free(p);   /* DOUBLE FREE - UNDEFINED BEHAVIOR */
```

On the 1st free(p), p's region is added to the free list. On the 2nd free(p), the allocator adds the SAME region to the free list again

→ corrupts its internal data (heap corruption).

Consequences:

- May crash immediately with glibc detected double free or corruption
- May NOT crash immediately, but silently corrupts the heap, crashing later at an unrelated malloc()/free() call
- A classic security vulnerability — controlled memory overwrite that can lead to arbitrary code execution

```
free(p);
p = NULL;
free(p);   /* SAFE */
```

6. Compilation Pipeline

4 stages

```
source.c → [Preprocessor] → source.i → [Compiler] → source.s →
[Assembler] → source.o → [Linker] → executable
```

Stage	What it does	Command to view separately
Preprocessing	Handles `#include`, `#define`, macros	`gcc -E file.c -o file.i`
Compilation	C → Assembly	`gcc -S file.c -o file.s`
Assembly	Assembly → Object code (not yet linked)	`gcc -c file.c -o file.o`
Linking	Combines `.o` files + libraries → executable	`gcc file.o -o program`

6.1. Compiling multiple files

```
gcc -c main.c -o main.o
gcc -c utils.c -o utils.o
gcc main.o utils.o -o program
```

6.2. Important flags

```
gcc -Wall -Wextra -g -o program main.c utils.c
```

- `-Wall -Wextra`: enable warnings
- `-g`: embed debug symbols → **required to use GDB**
- `-c`: compile only, don't link
- `-o`: set the output name

6.3. Static vs Dynamic Linking

- **Static**: the library is embedded into the executable (`.a`)
- **Dynamic**: linked at runtime (`.so`), a smaller executable but requires the library to already be present on the running machine

7. Basic Data Structures

7.1. Singly Linked List

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node
{
    int data;
    struct Node *next;
} Node;
Node* createNode(int value)
{
    Node *newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->data = value;
```

```
    newNode->next = NULL;
    return newNode;
}
void insertHead(Node **head, int value)
{
    Node *newNode = createNode(value);
    newNode->next = *head;
    *head = newNode;
}
void insertTail(Node **head, int value)
{
    Node *newNode = createNode(value);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    Node *temp = *head;
    while (temp->next != NULL)
    {
        temp = temp->next;
    }
    temp->next = newNode;
}
void deleteNode(Node **head, int value)
{
    Node *temp = *head;
    Node *prev = NULL;
    if (temp != NULL && temp->data == value)
    {
        *head = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != value)
    {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL)
```

```
    {
        printf("Value %d not found in the list.\n", value);
        return;
    }
    prev->next = temp->next;
    free(temp);
}
void traverse(Node *head)
{
    Node *temp = head;
    if (temp == NULL) {
        printf("List is empty.\n");
        return;
    }
    while (temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
void freeList(Node *head)
{
    Node *temp;
    while (head != NULL)
    {
        temp = head;
        head = head->next;
        free(temp);
    }
}
int main()
{
    Node *head = NULL;
    insertTail(&head, 10);
    insertTail(&head, 20);
    insertTail(&head, 30);
    insertHead(&head, 5);
    printf("List after insertion: ");
    traverse(head);
```

```
    deleteNode(&head, 20);
    printf("List after deleting 20: ");
    traverse(head);
    deleteNode(&head, 100);
    freeList(head);
    head = NULL;
    printf("Successfully freed the entire list.\n");
    return 0;
}
```

Output:

```
List after insertion: 5 -> 10 -> 20 -> 30 -> NULL
List after deleting 20: 5 -> 10 -> 30 -> NULL
Value 100 not found in the list.
Successfully freed the entire list.
```

- createNode(int value) – Create a new node: Dynamically allocate memory (using malloc) to create a new node containing the value value.
- insertHead(Node **head, int value) – Insert at the head: Add a new element to the beginning of the list.
- insertTail(Node **head, int value) – Insert at the tail: Add a new element to the end of the list.
- deleteNode(Node **head, int value) – Delete a node by value: Find and delete the first node where data == value.
- traverse(Node *head) – Traverse and print the list: Print all data in the list from head to tail to the screen.
- freeList(Node *head) – Free all memory: Deallocate all Heap memory allocated during list creation to prevent memory leaks.

7.2. Doubly Linked List

Each node also has a pointer that points **back to the previous node** — allowing traversal in both directions.

```
typedef struct DNode
{
    int data;
    struct DNode *next;
    struct DNode *prev;
} DNode;
```

```
void insert_front_d(DNode **head, int val)
{
    DNode *n = malloc(sizeof(DNode));
    n->data = val;
    n->prev = NULL;
    n->next = *head;

    /* update the old node's prev */
    if (*head != NULL)
    {
        (*head)->prev = n;
    }
    *head = n;
}
```

	Singly Linked List	Doubly Linked List
Traversal	1 direction only (forward)	2 directions (forward and backward)
Memory per node	1 pointer (`next`)	2 pointers (`next`, `prev`)
Deleting a node (address known)	Needs to traverse to find the preceding node	Can delete directly via `prev`

7.3. Array-based Structures

Stack (LIFO — Last In First Out)

```
#define MAX 100

typedef struct
{
    int items[MAX];
    int top;
} Stack;

void push(Stack *s, int val)
{
    s->items[++s->top] = val;
}

int pop(Stack *s)
```

```
{
    return s->items[s->top--];
}
```

Queue (FIFO — First In First Out)

```
typedef struct
{
    int items[MAX];
    int front, rear;
} Queue;

void enqueue(Queue *q, int val)
{
    q->items[q->rear++] = val;
}

int dequeue(Queue *q)
{
    return q->items[q->front++];
}
```

8. Function Pointers & Callback-Style APIs

8.1. Declaring and using function pointers

```
int add(int a, int b)
{
    return a + b;
}

int sub(int a, int b)
{
    return a - b;
}

/* declares a function pointer, type: takes (int,int), returns int */
int (*operation)(int, int);
```

```
operation = add;
printf("%d\n", operation(2, 3));   /* 5 */

operation = sub;
printf("%d\n", operation(5, 2));   /* 3 */
```

Declaration `int (*fp)(int, int)`: `fp` is a pointer, pointing to a function that takes 2 `int` parameters and returns `int`.

The parentheses around `*fp` are required — if omitted (`int *fp(int, int)`), it becomes a declaration of a function returning `int *`.

8.2. Array of function pointers

```
int (*ops[4])(int, int) = {add, sub, mul, div_};

int result = ops[0](10, 5);   /* calls add(10, 5) */
```

→ This pattern is very common in kernel drivers (`struct file_operations` holds function pointers like `.open`, `.read`, `.write`...).

8.3. Callback — passing a function as a parameter

```
/* function that takes another function pointer as a param (callback) */
void process_array(int *arr, int size, void (*callback)(int))
{
    for (int i = 0; i < size; i++)
    {
        callback(arr[i]);      /* call the callback for each element */
    }
}

void print_val(int x)
{
    printf("%d ", x);
}

void print_square(int x)
{
    printf("%d ", x * x);
}
```

```
int main()
{
    int arr[] = {1, 2, 3, 4};
    process_array(arr, 4, print_val);      /* prints: 1 2 3 4 */
    process_array(arr, 4, print_square);   /* prints: 1 4 9 16 */
}
```

Core idea: process_array doesn't need to know what to do with each element — that logic is "injected" via the function pointer passed at call time. This lets one generic function behave flexibly without modifying its own code.

8.4. `typedef` for function pointers

```
typedef void (*Callback)(int);

void process_array(int *arr, int size, Callback cb)
{
    for (int i = 0; i < size; i++)
    {
        cb(arr[i]);
    }
}
```

Real-world application	Role of the function pointer
Signal handler (`signal()`, `sigaction()`)	Registers a callback function that the kernel calls when a signal occurs
I/O multiplexing (`select`/`poll`/`epoll` combined with threads)	Each ready file descriptor → calls the corresponding handling callback
Socket event loop	Registers a separate handler for each type of event (connect, read, disconnect)
Kernel module (`struct file_operations`)	A table of function pointers for `.open`, `.read`, `.write`, `.release`

```
#include <signal.h>

void handle_sigint(int sig)
{
    /* this is exactly a "callback" */
    printf("Caught Ctrl+C (signal %d)\n", sig);
}
```

```
int main()
{
    /* register the callback for the SIGINT signal */
    signal(SIGINT, handle_sigint);
    while (1)
    {
        /* program runs, waiting for the signal */
    }
}
```

9. Pointer Size & Relation to OS/Architecture

9.1. What does `sizeof(pointer)` depend on?

```
int *p;
printf("%zu\n", sizeof(p));
```

The result doesn't depend on the type of data the pointer points to — it depends on the system's architecture/addressing mode:

System	Pointer size	Max address space
32-bit (x86, ARM 32-bit)	4 bytes	2^32 = 4 GB
64-bit (x86-64, ARM64/AArch64)	8 bytes	2^64

```
printf("%zu\n", sizeof(int*));     /* 8 on 64-bit, 4 on 32-bit */
printf("%zu\n", sizeof(char*));    /* same as int* - stores address */
printf("%zu\n", sizeof(double*));  /* same as int* */
```

9.2. Direct connection to the operating system

Whether an OS/program is "32-bit" or "64-bit" is defined by the pointer size, not the CPU. A program compiled in 32-bit mode uses 4-byte pointers even on a 64-bit CPU — limited to 4GB of virtual memory, even if the machine has more RAM.

Effect on structs containing pointers: sizeof(struct) differs between 32-bit and 64-bit systems — matters when sharing binary data/structs across architectures.

```
struct Example
{
    char c;
    int *p;
};
/* On 64-bit: sizeof = 16 (1 byte c + 7 padding + 8-byte pointer) */
/* On 32-bit: sizeof = 8  (1 byte c + 3 padding + 4-byte pointer) */
```

```
int *p = malloc(sizeof(int));
int addr_as_int = (int)p;   /* On 64-bit - truncates half address */

/* correct way to cast a pointer to an integer: */
intptr_t addr = (intptr_t)p;
```

10. Endianness

Byte order when storing multi-byte numbers in memory

```
int x = 0x12345678;   /* a 4-byte int */
```

Big-Endian: the most significant byte is stored at the LOWEST address

```
Address:  1000  1001  1002  1003
Value:    0x12  0x34  0x56  0x78
```

Little-Endian: the least significant byte is stored at the LOWEST address

```
Address:  1000  1001  1002  1003
Value:    0x78  0x56  0x34  0x12
```

How to check endianness

```
#include <stdio.h>

int main()
{
    unsigned int x = 1;

    /* points to the first byte of x in memory */
```

```
    char *c = (char*)&x;

    if (*c == 1)
    {
        /* least significant byte (value 1) is at first address */
        printf("Little-Endian\n");
    }
    else
    {
        printf("Big-Endian\n");
    }
}
```

How it works: `x = 1` sets the least-significant byte to `0x01`. On Little-Endian, that byte sits at the FIRST address → `*c` reads `1`. On Big-Endian, the first byte is `0x00` → `*c` reads `0`.

- **Network byte order is always Big-Endian** (RFC standard) — when sending integers over a socket (port, IP address), convert from "host byte order" (usually Little-Endian on x86) to "network byte order" before sending, and convert back when receiving.

```
#include <arpa/inet.h>

uint16_t port = 8080;

/* Host TO Network Short - convert to big-endian before sending */
uint16_t port_network = htons(port);

/* Network TO Host Short - convert back when receiving */
uint16_t port_host = ntohs(port_network);

uint32_t ip = 0xC0A80001;   /* e.g. 192.168.0.1 */

/* Host TO Network Long (for 4-byte numbers) */
uint32_t ip_network = htonl(ip);
```

If forget to convert: values like port numbers get misread on a machine with different endianness — a bug that stays hidden until two systems of different architectures talk to each other (e.g., x86 with a Big-Endian ARM/embedded device).

- Same risk applies when reading binary files or casting a `struct` onto raw memory from another device with different endianness.

Revision #12
Created 2026-08-18 10:23:52 UTC by Huyen Nguyen
Updated 2026-09-14 02:09:09 UTC by Huyen Nguyen